

Practical Set – 10 (JAVA FX)

Practical 52

Problem Statement:

Write a JavaFX program that displays five Text nodes vertically (stacked). For each Text:

- Use font Times New Roman, bold + italic, size 22 px.
- Assign a random color and random opacity (between 0.3 and 1.0) to each text.
- Center the texts horizontally in the window and add spacing between them.

UI / Classes to use: VBox, Text, Font, Color, Random.

On launch the window shows five vertically arranged lines (e.g., “Text 1”, ... “Text 5”) each with different color & opacity and same font style.

Code:

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontPosture;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;
import javafx.stage.Stage;
import java.util.Random;

public class VerticalTextApp extends Application {

    @Override
    public void start(Stage primaryStage) {
        System.out.println("maitri chauhan, 25039310003");
```

```
VBox root = new VBox(20);
root.setAlignment(Pos.CENTER);

Random random = new Random();

Font tnrFont = Font.font("Times New Roman", FontWeight.BOLD,
FontPosture.ITALIC, 22);

for (int i = 1; i <= 5; i++) {
    Text text = new Text("Text " + i);
    text.setFont(tnrFont);

    double r = random.nextDouble();
    double g = random.nextDouble();
    double b = random.nextDouble();
    double opacity = 0.3 + (1.0 - 0.3) * random.nextDouble();

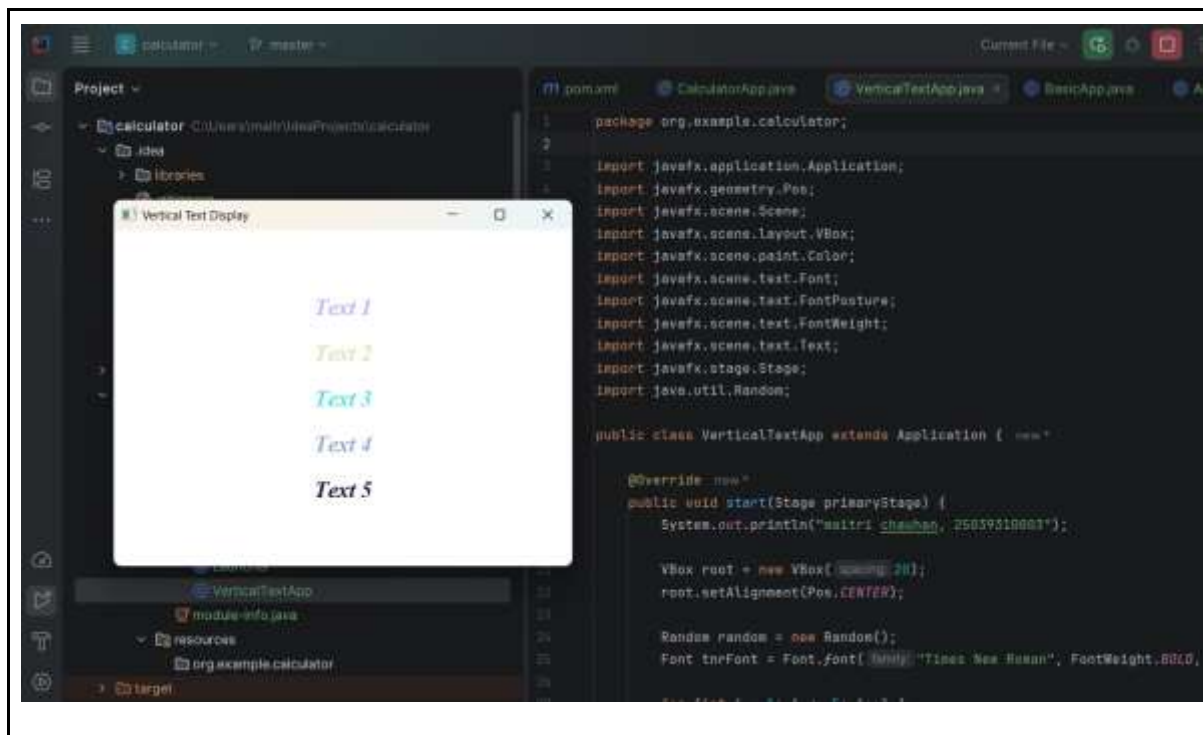
    text.setFill(new Color(r, g, b, opacity));
    root.getChildren().add(text);
}

Scene scene = new Scene(root, 400, 400);
primaryStage.setTitle("Vertical Text Display");
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
```

```
    launch(args);  
}  
}
```

Output:



Problem Statement:

Create a JavaFX app that shows a circle which continuously bounces (left↔right) inside the scene. Provide:

- A Start and Stop button.
- A Slider to control the ball speed (min: slow, max: fast).
- When window is resized, the ball should keep bouncing within new bounds.

UI / Classes to use: Pane or AnchorPane, Circle, TranslateTransition or AnimationTimer, Button, Slider.

Following behavior is expected:

- Press Start → ball moves horizontally and reverses on boundaries.
- Move Slider while running → ball speed updates.
- Press Stop → animation pauses.

Code:

```
import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Slider;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.stage.Stage;

public class BouncingBallApp extends Application {

    private double dx = 2;
```

```
private boolean running = false;

@Override
public void start(Stage primaryStage) {
    System.out.println("maitri chauhan, 25039310003");

    Pane pane = new Pane();
    Circle ball = new Circle(20, Color.DODGERBLUE);
    ball.setCenterX(50);
    ball.setCenterY(100);
    pane.getChildren().add(ball);

    Slider speedSlider = new Slider(1, 10, 2);
    speedSlider.setShowTickLabels(true);

    AnimationTimer timer = new AnimationTimer() {
        @Override
        public void handle(long now) {
            if (running) {
                double speed = speedSlider.getValue();
                ball.setCenterX(ball.getCenterX() + (dx > 0 ? speed : -speed));

                if (ball.getCenterX() + ball.getRadius() >= pane.getWidth() ||
ball.getCenterX() - ball.getRadius() <= 0) {
                    dx *= -1;
                }
            }
        }
    }
}
```

```
};

Button startBtn = new Button("Start");
startBtn.setOnAction(e -> {
    running = true;
    timer.start();
});

Button stopBtn = new Button("Stop");
stopBtn.setOnAction(e -> running = false);

HBox controls = new HBox(15, startBtn, stopBtn, speedSlider);
controls.setAlignment(Pos.CENTER);
controls.setStyle("-fx-padding: 10; -fx-background-color: #f4f4f4;");

BorderPane root = new BorderPane();
root.setCenter(pane);
root.setBottom(controls);

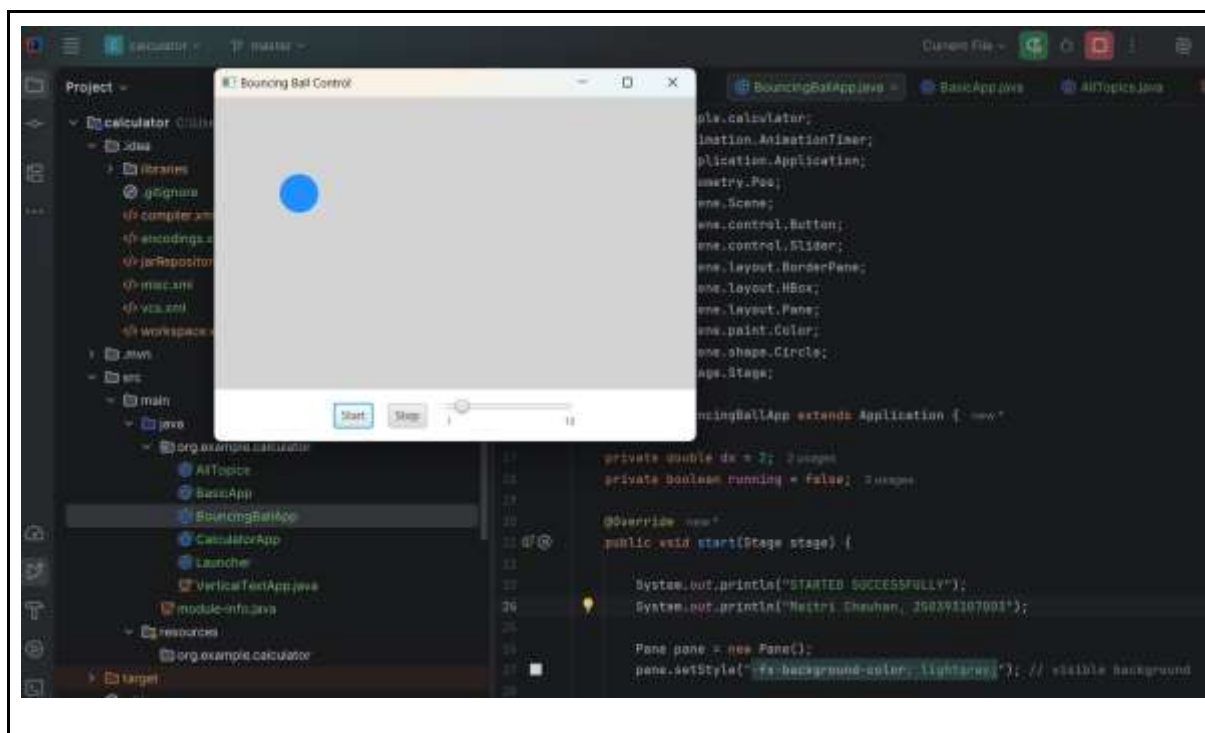
Scene scene = new Scene(root, 500, 300);
primaryStage.setTitle("Bouncing Ball Control");
primaryStage.setScene(scene);
primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}
```

```
}

```

Output:



Practical 54

Problem Statement:

Design a registration form UI with fields: Roll_NO (numeric), Name, Age (numeric), Email and a Submit button. Requirements:

· Validate inputs on submit:

1. RollNo and Age must be integers.
 2. Email must contain @ and . (basic check).
- On success show a confirmation Alert with entered data.
 - On validation failure show an error alert describing the issue.

UI / Classes to use: GridPane, TextField, Button, Alert, FileChooser, FileWriter/BufferedWriter.

Code:

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class RegistrationForm extends Application {

    @Override
    public void start(Stage primaryStage) {

        System.out.println("maitri chauhan, 25039310003");

        GridPane grid = new GridPane();
        grid.setAlignment(Pos.CENTER);
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new Insets(25, 25, 25, 25));

        TextField rollField = new TextField();
        TextField nameField = new TextField();
        TextField ageField = new TextField();
        TextField emailField = new TextField();
```



```
grid.add(new Label("Roll No:"), 0, 0);
grid.add(rollField, 1, 0);
grid.add(new Label("Name:"), 0, 1);
grid.add(nameField, 1, 1);
grid.add(new Label("Age:"), 0, 2);
grid.add(ageField, 1, 2);
grid.add(new Label("Email:"), 0, 3);
grid.add(emailField, 1, 3);

Button submitBtn = new Button("Submit");
grid.add(submitBtn, 1, 4);

submitBtn.setOnAction(e -> {
    String roll = rollField.getText();
    String name = nameField.getText();
    String age = ageField.getText();
    String email = emailField.getText();

    StringBuilder errorMessage = new StringBuilder();

    try {
        Integer.parseInt(roll);
    } catch (NumberFormatException ex) {
        errorMessage.append("- Roll No must be an integer.\n");
    }

    try {
        Integer.parseInt(age);
```

```
        } catch (NumberFormatException ex) {
            errorMessage.append("- Age must be an integer.\n");
        }

        if (!email.contains("@") || !email.contains(".")) {
            errorMessage.append("- Email must contain '@' and '.'.\n");
        }

        if (errorMessage.length() > 0) {
            Alert errorAlert = new Alert(Alert.AlertType.ERROR);
            errorAlert.setTitle("Validation Error");
            errorAlert.setHeaderText("Invalid Input Detected");
            errorAlert.setContentText(errorMessage.toString());
            errorAlert.showAndWait();
        } else {
            Alert successAlert = new Alert(Alert.AlertType.INFORMATION);
            successAlert.setTitle("Success");
            successAlert.setHeaderText("Registration Confirmed");
            successAlert.setContentText("Roll No: " + roll + "\nName: " + name + "\nAge: "
+ age + "\nEmail: " + email);
            successAlert.showAndWait();
        }
    });

    Scene scene = new Scene(grid, 400, 300);
    primaryStage.setTitle("Student Registration Form");
    primaryStage.setScene(scene);
    primaryStage.show();
```

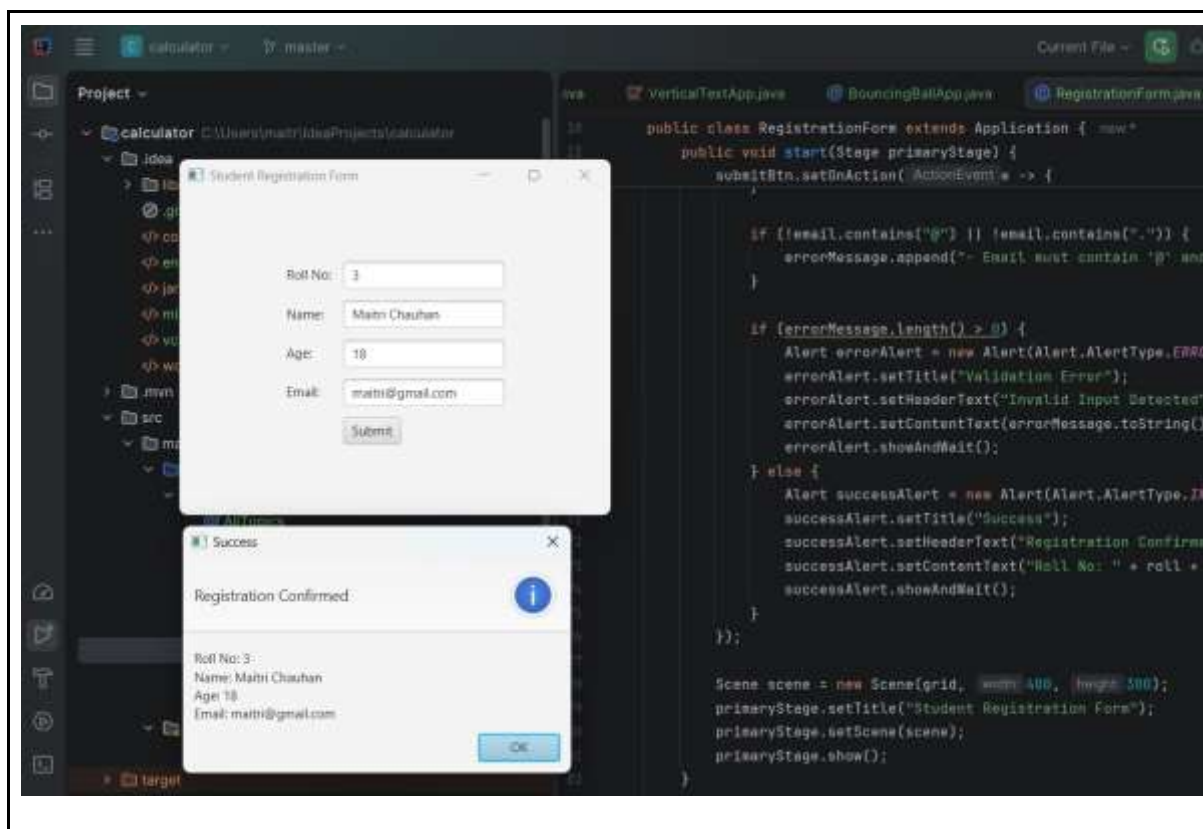
```

    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

Output:



Practical 55

Problem Statement:

Write a JavaFX program that displays a bar chart to represent the percentage distribution of overall grades using Rectangle shapes.

- Projects: 20%, displayed in Red

- Quizzes: 10%, displayed in Blue
- Midterm Exams: 30%, displayed in Green
- Final Exam: 40%, displayed in Orange

Requirements:

1. Each category should be displayed with a labeled bar.
2. Bars should be proportional in height to the percentage.
3. Use the Rectangle class to create the bars.
4. Display the labels (e.g., "Projects — 20%") under each bar.
5. Arrange the bars horizontally in the scene using an HBox or Pane.

Code:

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class GradeBarChart extends Application {

    @Override
    public void start(Stage primaryStage) {

        System.out.println("maitri chauhan, 25039310003");

        HBox chartContainer = new HBox(30);
```

```
chartContainer.setAlignment(Pos.BOTTOM_CENTER);
chartContainer.setStyle("-fx-padding: 50;");

chartContainer.getChildren().addAll(
    createBar("Projects", 20, Color.RED),
    createBar("Quizzes", 10, Color.BLUE),
    createBar("Midterm", 30, Color.GREEN),
    createBar("Final Exam", 40, Color.ORANGE)
);

Scene scene = new Scene(chartContainer, 600, 400);
primaryStage.setTitle("Grade Distribution Bar Chart");
primaryStage.setScene(scene);
primaryStage.show();
}

private VBox createBar(String category, int percentage, Color color) {
    int scaleFactor = 5;
    Rectangle bar = new Rectangle(60, percentage * scaleFactor);
    bar.setFill(color);

    Text label = new Text(category + " — " + percentage + "%");
    label.setFont(Font.font("Arial", 12));

    VBox barGroup = new VBox(10);
    barGroup.setAlignment(Pos.BOTTOM_CENTER);
    barGroup.getChildren().addAll(bar, label);
```

```
        return barGroup;
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:

